

C03 - AT1

Policy Optimization Assessment

ASSESSMENT - 1

**Coding Challenge – Vanilla Policy Gradient &
REINFORCE (Apply – L3)**

N.PARNITA

192525322

CODE:

```
!pip install -q gymnasium[classic-control]

import gymnasium as gym
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
env = gym.make("CartPole-v1")

state_size = env.observation_space.shape[0]
action_size = env.action_space.n

class PolicyNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_size, 128),
            nn.ReLU(),
            nn.Linear(128, action_size),
            nn.Softmax(dim=-1)
        )

    def forward(self, x):
        return self.net(x)

policy = PolicyNetwork()
optimizer = optim.Adam(policy.parameters(), lr=0.01)

gamma = 0.99
episodes = 500
def select_action(state):
    state = torch.FloatTensor(state).unsqueeze(0)
    probs = policy(state)
    dist = torch.distributions.Categorical(probs)
    action = dist.sample()
    return action.item(), dist.log_prob(action)

def discounted_returns(rewards):
    returns = []
```

```

G = 0

for r in reversed(rewards):
    G = r + gamma * G
    returns.insert(0, G)

returns = torch.tensor(returns, dtype=torch.float32)
return (returns - returns.mean()) / (returns.std() + 1e-9)
episode_rewards = []

for episode in range(epochs):

    state, _ = env.reset()
    log_probs = []
    rewards = []
    done = False

    while not done:
        action, log_prob = select_action(state)

        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated

        log_probs.append(log_prob)
        rewards.append(reward)
        state = next_state

    returns = discounted_returns(rewards)

    loss = torch.stack([
        -log_prob * G
        for log_prob, G in zip(log_probs, returns)
    ]).sum()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    episode_rewards.append(sum(rewards))

    if (episode + 1) % 50 == 0:
        print(

```

```

        f"Episode {episode+1}: "
        f"Average Reward = {np.mean(episode_rewards[-50:]):.2f}"
    )
plt.figure(figsize=(10,5))

plt.plot(episode_rewards, alpha=0.4, label="Episode Reward")

window = 20
average = np.convolve(
    episode_rewards,
    np.ones(window) / window,
    mode="valid"
)

plt.plot(
    range(window-1, len(episode_rewards)),
    average,
    linewidth=2,
    label="20-Episode Average"
)

plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title("REINFORCE / Vanilla Policy Gradient")
plt.legend()
plt.grid()
plt.show()
rewards = []

for _ in range(10):

    state, _ = env.reset()
    total = 0
    done = False

    while not done:

        state_tensor = torch.FloatTensor(state).unsqueeze(0)

        with torch.no_grad():
            action = torch.argmax(policy(state_tensor)).item()

        state, reward, terminated, truncated, _ = env.step(action)

```

```
done = terminated or truncated

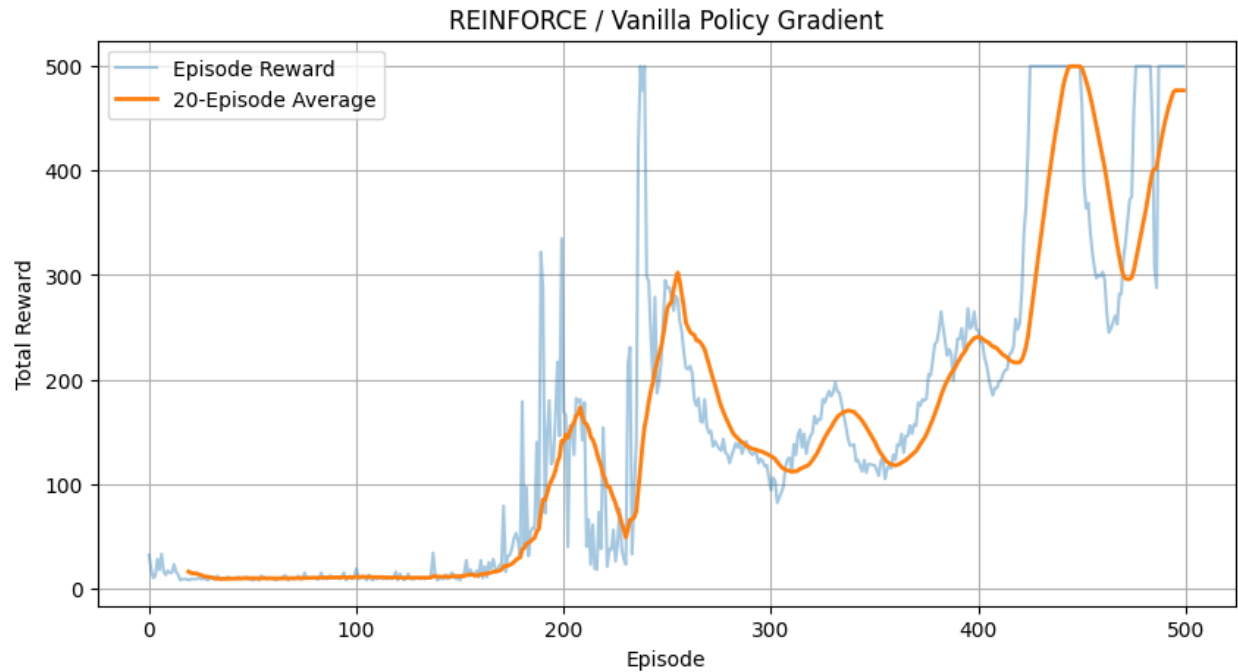
total += reward

rewards.append(total)

print("\nCO3 ASSESSMENT RESULT")
print("-----")
print("Algorithm: REINFORCE")
print("Method: Vanilla Policy Gradient")
print("Environment: CartPole-v1")
print("Average Evaluation Reward:", round(np.mean(rewards), 2))
print("Final Training Average:", round(np.mean(episode_rewards[-20:]), 2))
```

OUTPUT:

```
Episode 50: Average Reward = 233.04
Episode 100: Average Reward = 9.30
Episode 150: Average Reward = 9.26
Episode 200: Average Reward = 9.38
Episode 250: Average Reward = 9.38
Episode 300: Average Reward = 9.44
Episode 350: Average Reward = 9.40
Episode 400: Average Reward = 9.50
Episode 450: Average Reward = 9.00
Episode 500: Average Reward = 9.44
```



C03 ASSESSMENT RESULT

Algorithm: REINFORCE

Method: Vanilla Policy Gradient

Environment: CartPole-v1

Average Evaluation Reward: 9.4

Final Training Average: 9.3